begins. Typically, you will configure the server to first check if the resource exists on disk, and if not, then fall back to a single file (by convention, this file is usually `index.php`). What benefits could this bring? Consider a more complex resource path, one that might include a more meaningful filename and a query string with metadata used to locate a record in a database, such as `/article.php?id=12345`. Without needing to worry about what code lives inside `article.php`, we can guess that it is targeting a specific `article` entity indicated by the `id` query parameter. To differentiate between CRUD (Create, Read, Update, Delete) operations, you might put the relevant functionality into differently named PHP files: `read_article.php`, `delete_article.php`, and so forth. The trouble here is that we're accumulating URLs, and they're not particularly pretty. What if we could replace this…

- `/read_article?id=123`
- `/create_article`
- `/delete_article?id=123`
- `/update_article?id=123`

… with this:

- `GET /article/123`
- `POST /article`
- `PUT /article/123`
- `DELETE /article/123`
- `PATCH /article/123`

Hopefully we can all agree that the second URL scheme is far tidier; it's also more SEO-friendly, which will make any members of your team with the word "product" in their job title much happier! The key innovation is that we've removed the `?id=x` query parameter and replaced it with a path component. For this to work we need to move the logic of mapping the HTTP resource path out of the web server and into our application, where we have far more freedom. To achieve this, you'll need to configure the web server to fall back to a single file when it can't map the path in the request to a filepath on the server.